

All About Data

USP 410/510 | Spring 2026

Dr. Liming Wang

Portland State University

Part 1: The Data Preparation Pipeline

Based on Yu & Barter, *Veridical Data Science*, Chapter 4

Why Data Preparation Matters

You will spend **most of your time** here

- Real-world data is messy, incomplete, and inconsistent
- Analysis results are only as good as the data going in
- Data cleaning decisions are **subjective** — they encode assumptions
- Documenting your choices is part of responsible data science

"Data preparation is more than a technical exercise — the choices made during cleaning and preprocessing can fundamentally shape the conclusions of an analysis."

— Yu & Barter

The Data Preparation Pipeline

1. **Data collection** — Where does the data come from? What format?
2. **Data cleaning** — Fix errors, handle missing values, standardize formats
3. **Data preprocessing** — Transform cleaned data for a specific analysis
4. **Exploratory Data Analysis** — Summaries and visualizations to understand patterns

These stages are **not linear** — expect to cycle back as you learn more about your data

Common Data Quality Issues

- **Missing values** — blanks, NaN , sentinel values like -999 or Unknown
- **Duplicates** — repeated rows from joins or data entry
- **Inconsistent formats** — "Portland" vs "portland" vs "PORTLAND, OR"
- **Wrong types** — numbers stored as strings, dates as text
- **Outliers** — legitimate extremes vs data entry errors
- **Structural problems** — data spread across multiple files or tables

Each of these requires a **decision**, not just a function call

Tidy Data

A framework for structuring datasets (Hadley Wickham, 2014)

- Each **variable** is a column
- Each **observation** is a row
- Each **value** is a cell

Why tidy?

- Consistent structure makes analysis code simpler
- Most Python tools (pandas, seaborn, plotly) expect tidy data
- Easier to merge, filter, group, and visualize

Much of data cleaning is about getting messy data **into** tidy form

Part 2: Reading Data into Python

Based on McKinney, *Python for Data Analysis*, Chapter 6

pandas: The Core Library

pandas is Python's primary tool for tabular data

```
import pandas as pd
```

- **DataFrame** — a table (rows and columns), like a spreadsheet
- **Series** — a single column
- Handles reading/writing many file formats
- Built-in tools for cleaning, filtering, grouping, and merging

Reading Data

pandas has `pd.read_*` functions for every common format

```
df = pd.read_csv('crashes.csv')           # CSV
df = pd.read_excel('report.xlsx')          # Excel
df = pd.read_json('data.json')             # JSON
df = pd.read_parquet('data.parquet')       # Parquet
```

For ODOT crash data — an ESRI **File Geodatabase** (`.gdb`):

```
import geopandas as gpd
crashes = gpd.read_file('data/Statewide_Crashes_2023.gdb', layer='CRASH')
```

ODOT Crash Data Structure

A geodatabase is like an Excel workbook with multiple **layers** (sheets)

LAYER	CONTAINS
CRASH	One row per crash event
VHCL	One row per vehicle involved
PARTIC	One row per participant (driver, pedestrian, etc.)
CRASH_GEOLOC	Crash locations with geometry

These tables are linked by `CRASH_ID` — use `pd.merge()` to combine them

First Look at Your Data

After loading, **always** inspect immediately

```
df.shape      # (rows, columns)
df.dtypes     # data types per column
df.head()     # first 5 rows
df.info()     # types, non-null counts, memory
df.describe() # summary statistics
```

Why?

- Catch wrong types early (e.g., ZIP codes read as integers)
- Spot unexpected missing values
- Understand the scale of the data before writing analysis code

Part 3: Data Cleaning

Based on McKinney, *Python for Data Analysis*, Chapter 7

The Cleaning Toolkit

TASK	KEY FUNCTIONS
Missing values	<code>isna()</code> , <code>dropna()</code> , <code>fillna()</code>
Duplicates	<code>uplicated()</code> , <code>drop_duplicates()</code>
Type conversion	<code>astype()</code> , <code>pd.to_datetime()</code>
String cleanup	<code>.str.upper()</code> , <code>.str.strip()</code> , <code>.str.contains()</code>
Filtering rows	Boolean indexing, <code>.query()</code>

Handling Missing Data

Strategies:

- **Drop** rows/columns — when missingness is rare and random
- **Fill** with a value (0, mean, median) — when you can justify a default
- **Flag** and keep — when missingness itself is informative

Key question: Is the data *missing at random* or is the missingness informative?

For example, if crash severity is missing only for minor incidents, dropping those rows biases your analysis toward severe crashes

Common Type Problems

Getting types right is essential for correct analysis

SYMPTOM	LIKELY CAUSE	FIX
Numbers with commas	Read as string	<code>.str.replace(',', ' ').astype(float)</code>
Dates as strings	Not parsed on read	<code>pd.to_datetime(df['col'])</code>
ZIP codes like <code>7201</code>	Leading zero dropped	Read with <code>dtype={'ZIP': str}</code>
Categories as strings	Wastes memory	<code>.astype('category')</code>

Tip: String inconsistencies are one of the most common data quality issues

```
# How many unique spellings of city names?
df['CITY'].nunique()
df['CITY'].value_counts().tail(20) # check the rare values
```

Part 4: Combining and Reshaping Data

Based on McKinney, *Python for Data Analysis*, Chapter 8

Merging (Joins)

Combine two DataFrames on a shared key — like SQL joins

```
merged = pd.merge(crashes, vehicles, on='CRASH_ID', how='left')
```

Join types:

TYPE	KEEPS
inner	Only rows with matching keys in both
left	All rows from left + matches from right
right	All rows from right + matches from left
outer	All rows from both

After every merge, check: did the number of rows change unexpectedly?

```
print(f"Crashes: {len(crashes)}, Vehicles: {len(vehicles)}, Merged: {len(merged)}")
```

Other Combining Operations

Concatenation — stack DataFrames vertically or horizontally

```
all_years = pd.concat([crashes_2022, crashes_2023, crashes_2024])
```

Grouping and aggregation — split-apply-combine

```
df.groupby('CRASH_YR').agg(  
    total_crashes=('CRASH_ID', 'count'),  
    avg_vehicles=('TOT_VHCL_CNT', 'mean')  
)
```

Reshaping — transform between wide and long formats

- `pd.melt()` — wide to long (columns become rows)
- `df.pivot_table()` — long to wide (rows become columns)
- Most plotting libraries prefer **long** format

Hands-on Notebook

All of today's code is in the companion notebook:

week3_data_processing.ipynb

1. Reading CSV, Excel, and geodatabase files
2. Inspecting data with `shape` , `dtypes` , `head` , `info` , `describe`
3. Cleaning: missing values, duplicates, types, strings
4. Filtering and selecting subsets
5. Merging, concatenating, grouping, reshaping
6. End-to-end workflow with ODOT crash data
7. Saving your work and common pitfalls

Key Takeaways

- **Data preparation is most of the work** — plan time for it
- **Always inspect your data first** — `shape` , `dtypes` , `head` , `info`
- **Cleaning decisions are analytical decisions** — document them
- **pandas is your main tool** — `read_*` , `merge` , `groupby` , `to_*`
- **Check your work at every step** — row counts, types, missing values
- **Save cleaned data** — separate raw from processed

For Next Week

Read:

- Yu & Barter, Chapter 3: Setting Up Your Data Science Project
- Turrell, Workflow chapters

Due this week:

- DataCamp DC1: Introduction to Python (due today, 04/16)
- Project idea (due today, 04/16)

Coming up:

- **Assignment 1** (Exploring ODOT Crash Data) — due W4 (04/23)
- Start **DataCamp DC2: Data Manipulation with pandas** (due W5)